GaitGuard: Real-Time Gait & Posture-Based Fatigue Monitor

via On-Device IMU Sensing and Optimized Temporal CNN

Mobile Computing and Its Applications (4190.406B) — Spring 2026 — Prof. Youngki Lee

Dong Kun Moon (2020-13236) — College of Liberal Studies (Department of Business Administration / Computer Science)
dongkun1115@snu.ac.kr · Solo Project

Goal & Usage Scenario

Problem. Physical fatigue changes how people walk before they consciously notice it: cadence becomes irregular, stride symmetry decreases, trunk tilt increases, and lateral sway becomes unstable. These changes are especially important for elderly people living independently, athletes at risk of overtraining, and post-surgery rehabilitation patients. However, existing gait-analysis systems usually require expensive wearable sensors, force plates, treadmill laboratories, or camera-based pose estimation systems, making them unsuitable for daily mobile use.

Proposed Solution. *GaitGuard* is a fully offline Android application that detects physical fatigue and fall risk using only the phone's built-in IMU sensors: the accelerometer and gyroscope. The phone is carried in a trouser pocket, shirt pocket, or belt clip. The app continuously processes 4-second IMU windows at 50 Hz and runs an optimized TensorFlow Lite model on-device. No camera, microphone, cloud server, or internet connection is required.

Usage Scenario. When the user starts a walking session, GaitGuard first records 5 seconds of stationary data to estimate the gravity vector and normalize the phone orientation. Then, while the user walks, the app displays:

- a real-time fatigue score from 0–100,
- stride symmetry score and left/right step balance,
- fall risk classification: Low, Medium, or High,
- live vertical acceleration waveform with detected step peaks,
- session timeline showing fatigue onset, recovery events, and fall-risk events.

If the fatigue score enters the amber zone, the app recommends rest through a vibration alert. If fall risk remains high or a true fall signature is detected, the app can send an SMS to a configured caregiver using Android's offline `SmsManager`.

Related Works

Smartphone-based activity recognition. The WISDM dataset showed that smartphone accelerometers can classify human activities such as walking, jogging, sitting, and standing with high reliability [1]. The UCI-HAR dataset also demonstrated that smartphone accelerometer and gyroscope signals contain enough information for robust human activity recognition [2]. GaitGuard builds on this idea, but moves beyond simple activity recognition toward fatigue estimation, stride symmetry analysis, and fall-risk classification.

Gait variability and fall risk. Prior gait studies show that increased stride-time variability, asymmetry, and irregular trunk movement are associated with aging, instability, and fall risk [3, 4]. In GaitGuard, these clinically meaningful indicators are extracted directly from IMU windows as hand-crafted features, including cadence, inter-stride interval coefficient of variation, harmonic ratio, trunk tilt, and trunk rotation amplitude.

Mobile model optimization. For mobile computing, accuracy alone is insufficient: the model must also be small, fast, and energy-efficient. Temporal Convolutional Networks (TCNs) are suitable for time-series data because dilated convolutions capture temporal patterns more efficiently than recurrent models [5]. INT8 quantization and pruning can reduce model size and latency with only marginal accuracy degradation [6, 7]. Therefore, GaitGuard compares a larger FP32 baseline model against a pruned and INT8-quantized TCN deployed with TensorFlow Lite.

Key Idea

The core idea is a **hybrid gait-analysis pipeline**: deterministic signal processing extracts interpretable gait features, while a lightweight DNN learns temporal patterns from raw IMU windows. This design makes the result both explainable and suitable for real-time mobile inference.

Signal Processing Branch

From each 4-second window, GaitGuard applies a 5 Hz low-pass filter to the vertical acceleration signal and detects steps using an adaptive threshold. It then computes:

- **Cadence:** steps per minute,
- **ISI coefficient of variation:** stride irregularity indicator,
- **Symmetry index:** odd/even step duration balance,
- **Spectral features:** dominant stride frequency, harmonic ratio, and signal energy,
- **Posture features:** trunk tilt and trunk rotation from gyroscope signals.

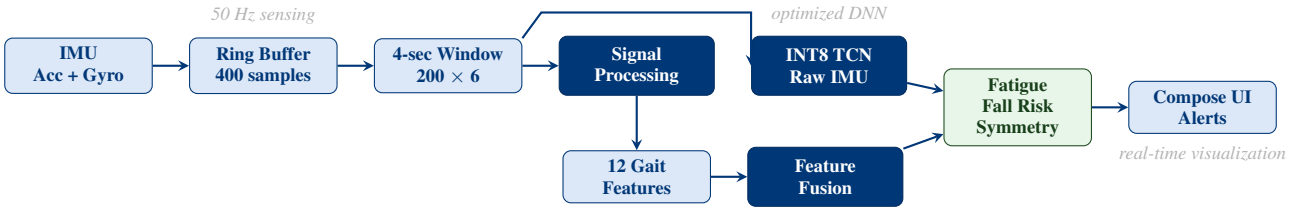


Figure 1: GaitGuard pipeline: IMU sensing → signal processing + optimized TCN → real-time fatigue, symmetry, and fall-risk visualization.

Since the phone cannot directly identify the left and right foot, alternating steps are treated as two legs. The symmetry index is computed as:

$$SI = \frac{|D_{\text{odd}} - D_{\text{even}}|}{0.5(D_{\text{odd}} + D_{\text{even}})} \times 100$$

A lower symmetry score indicates more imbalance and possible fatigue or instability.

DNN Branch and Composite Score

The DNN input is a raw IMU tensor of shape 200×6, combined with a 12-dimensional feature vector. The model is a Temporal Convolutional Network with dilated causal convolution blocks, depthwise separable convolutions, residual connections, and three output heads: fatigue probability, fall-risk class, and symmetry prediction.

The final fatigue score is a weighted fusion:

$$Score = 100 \times (0.55F_{TCN} + 0.20CV_{ISI} + 0.15H_{drop} + 0.10T_{tilt})$$

where F_{TCN} is the model fatigue output, CV_{ISI} is stride variability, H_{drop} is harmonic-ratio decline from baseline, and T_{tilt} is trunk-tilt deviation.

Green
Score 0–39: normal gait

Amber
Score 40–69: early fatigue

Red
Score 70–100: stop advised

Implementation Plan

System Architecture

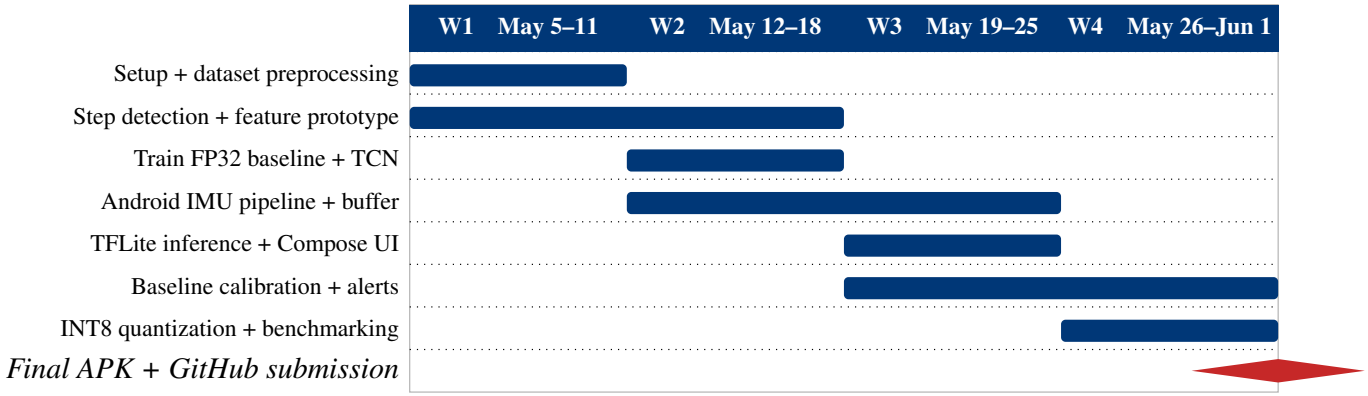
Component	Technology	Details
Android UI	Kotlin + Jetpack Compose	Real-time fatigue gauge, symmetry bar, waveform, session timeline
Sensing pipeline	Android SensorManager	50 Hz accelerometer + gyroscope using `SENSOR_DELAY_GAME`
Buffering	Circular ring buffer	400 samples, shared thread-safe IMU window storage
Signal processing	Kotlin implementation	Step detection, FFT features, trunk tilt, symmetry index
ML inference	TensorFlow Lite 2.14	INT8 TCN model with GPU delegate on Samsung Galaxy S21
Storage	Android Room + Preferences	Session history, personal baseline profile, CSV export
Alert system	Vibrator + SmsManager	Fatigue alerts, sustained-risk SMS, emergency fall SMS

Model Optimization and Evaluation

The baseline model will be a larger FP32 sequence model, while the production model will use a pruned and INT8-quantized TCN. The public datasets used for training and evaluation are WISDM and UCI-HAR, with additional self-collected walking samples for terrain and fall-risk behavior. Evaluation will compare:

- **Model size:** FP32 baseline vs. INT8 TCN, target $>75\%$ reduction,
- **Latency:** average inference time per 4-second window on Galaxy S21, target <15 ms,
- **Accuracy degradation:** fall-risk F1 drop after optimization, target $<2\%$,
- **Application performance:** false alert rate, terrain classification accuracy, and end-to-end update latency.

Project Timeline



References

[1] Kwapisz, J.R., Weiss, G.M., & Moore, S.A. (2011). Activity recognition using cell phone accelerometers. *ACM SIGKDD Explorations*, 12(2), 74–82.

[2] Anguita, D., et al. (2013). A public domain dataset for human activity recognition using smartphones. *ESANN 2013*.

[3] Kang, H.G., & Dingwell, J.B. (2008). Separating the effects of age and walking speed on gait variability. *Gait & Posture*, 27(4), 572–577.

[4] Weiss, A., et al. (2013). Does the evaluation of gait quality during daily life provide insight into fall risk? *Neurorehabilitation and Neural Repair*.

[5] Bai, S., Kolter, J.Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv:1803.01271*.

[6] Jacob, B., et al. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *CVPR 2018*.

[7] Han, S., Pool, J., Tran, J., & Dally, W. (2015). Learning both weights and connections for efficient neural network. *NeurIPS 2015*.